# Lab Report

## Software Project Scheduling

**Project:** Utility Management System
**Prepared By:** Jasika Noor | **ID:** 22CSE002 | **Session:** 2022-2023 | **Course:** Software Engineering (Lab)
**Submission Date:** August 23, 2026

---

## 1. Introduction

This lab report applies software project scheduling principles to the **Utility Management System** project. The Utility Management System is an integrated, enterprise-grade web application designed to streamline multi-utility operations including electricity, water, and gas monitoring, automated meter reading, dynamic billing engine, online payment processing, GIS spatial mapping of infrastructure, outage/maintenance management, and customer complaint resolution. Proper project scheduling ensures that high-risk modules (such as real-time meter ingestion and billing calculation engines) are systematically planned, resource-allocated, and executed within a 15-week academic semester timeline using Work Breakdown Structure (WBS), PERT estimation, and Critical Path Analysis.

## 2. Steps in Software Project Scheduling (Applied to This Project)

### Step 1: Define Scope and Objectives

Develop an end-to-end Utility Management System that automates utility meter tracking (electricity, water, gas), generates monthly invoices with tiered pricing structures, supports online payment gateways, maps utility grids via GIS, tracks infrastructure maintenance/outages, and handles customer tickets with real-time reporting.

### Step 2: Create Work Breakdown Structure (WBS)

Decompose the project into 6 core phases: Requirement Analysis & Systems Architecture, Database Schema & Security Design, Core Backend Services & Meter Integration, Frontend UI & Dashboard Development, GIS & Payment Gateway Integration, and System Testing, Security Audit & Cloud Deployment.

### Step 3: Identify Task Dependencies

Establish logical task precedence — e.g., Automated Billing Engine relies on Database Schema & Meter Data models, Payment Gateway Integration depends on Billing API completion, and System Security Testing depends on complete frontend/backend integration.

### Step 4: Estimate Task Duration

Apply expert judgment and PERT 3-point estimation (Optimistic, Most Likely, Pessimistic) for complex, high-uncertainty tasks such as meter data ingestion algorithms, real-time consumption anomaly detection, and payment gateway webhooks.

### Step 5: Assign Resources

Allocate core specialized roles across the project timeline: Project Manager, Full-Stack / Backend Developer, Frontend UI/UX Developer, Database Administrator, GIS Specialist, QA/Security Tester, and DevOps Engineer.

### Step 6: Develop Schedule

Construct a detailed Gantt chart aligning tasks with a 15-week academic semester timeline, ensuring parallel execution of decoupled frontend and backend modules.

### Step 7: Monitor and Control

Track weekly milestone progress against planned duration, monitor API response latency, run regression tests, and adjust schedules if third-party integrations (e.g., payment gateway APIs) experience delays.

## 3. Work Breakdown Structure (WBS)

| Phase | Tasks |
|---|---|
| **Planning** | Feasibility study, stakeholder requirement gathering, utility domain analysis, SRS documentation |
| **Design** | System architecture, relational database schema (meters, bills, payments, customers, GIS), UI mockups |
| **Development** | Database setup, REST APIs (Meters, Bills, Payments, Complaints, Maintenance), Auth & Access Control |
| **Integration** | GIS spatial mapping integration, automated billing calculation engine, payment gateway API integration |
| **Testing** | Unit & integration testing, billing calculation accuracy verification, security audit, load testing |
| **Deployment** | Server configuration, database migration, production build deployment, user manual & handover |

## 4. Task Duration and Dependencies

| Task | Duration | Dependency |
|---|---|---|
| Requirement Analysis & SRS | 3 weeks | — |
| Database & Architecture Design | 2 weeks | Requirement Analysis |
| Core Backend & Meter APIs | 4 weeks | Database & Architecture Design |
| Billing & Payment Integration | 3 weeks | Core Backend & Meter APIs |
| Frontend UI & GIS Integration | 3 weeks | Core Backend & Meter APIs |
| Testing & Security Audit | 2 weeks | Billing, Payment & Frontend Integration |
| Deployment & Documentation | 1 week | Testing & Security Audit |

**Total estimated project duration: 15 weeks** (approx. one academic semester), utilizing parallel execution paths for frontend UI/GIS and billing engine integration.

## 5. Gantt Chart (Textual)

| Task | W1 | W2 | W3 | W4 | W5 | W6 | W7 | W8 | W9 | W10 | W11 | W12 | W13 | W14 | W15 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Requirements & SRS | ■ | ■ | ■ | | | | | | | | | | | | |
| Architecture & DB Design | | | | ■ | ■ | | | | | | | | | | |
| Core Backend & Meter APIs | | | | | | ■ | ■ | ■ | ■ | | | | | | |
| Billing & Payment Engine | | | | | | | | | | ■ | ■ | ■ | | | |
| Frontend UI & GIS Integration | | | | | | | | | | ■ | ■ | ■ | | | |
| Testing & Security Audit | | | | | | | | | | | | | ■ | ■ | |
| Deployment & Handover | | | | | | | | | | | | | | | ■ |

## 6. PERT Estimation Example

Program Evaluation and Review Technique (PERT) is used to calculate Expected Task Duration (TE) under uncertainty using the formula **TE = (O + 4M + P) / 6**, where **O** = Optimistic time, **M** = Most Likely time, and **P** = Pessimistic time. This technique is applied to key complex tasks in the Utility Management System:

| Task | O (days) | M (days) | P (days) | TE (days) |
|---|---|---|---|---|
| Smart Meter Data Ingestion Pipeline Tuning | 4 | 7 | 14 | 7.7 |
| Automated Tiered Billing & Invoice Calculation Engine | 3 | 6 | 12 | 6.5 |
| GIS Spatial Map Layering & Utility Outage Tracking | 3 | 5 | 10 | 5.5 |
| Payment Gateway Webhooks & Reconciliation Engine | 2 | 4 | 9 | 4.5 |

## 7. Critical Path Analysis

**Critical Path:** Requirement Analysis & SRS → Database & Architecture Design → Core Backend & Meter APIs → Billing & Payment Engine → Testing & Security Audit → Deployment & Handover

**Total Critical Duration: 15 weeks.** The critical path represents the longest sequential sequence of dependent tasks. Any delay in core backend development or billing engine validation will directly push back the final deployment date of the Utility Management System.

## 8. Resource Allocation

| Role | Responsibility |
|---|---|
| **Project Manager** | Project planning, scheduling, risk mitigation, stakeholder reporting, and milestone tracking. |
| **Backend / DB Developer** | Database schema design, RESTful API implementation, authentication, and core business logic. |
| **Frontend / UX Developer** | Responsive web interface, admin dashboard, customer billing portal, and meter charts. |
| **GIS Specialist** | Spatial map integration, utility grid layout rendering, and outage zone mapping. |
| **QA / Security Tester** | Automated unit/integration testing, security audit, payment transaction validation, and load testing. |
| **DevOps / SysAdmin** | Cloud infrastructure deployment, CI/CD pipeline setup, database backups, and SSL/TLS security. |

## 9. Risks and Mitigation

| Risk | Mitigation Strategy |
|---|---|
| **Meter Data Ingestion Latency / Packet Loss** | Implement asynchronous message queue (e.g., RabbitMQ/Kafka) and local caching for meter readings. |
| **Billing Calculation Inaccuracies under Tiered Tariffs** | Develop rigorous automated unit test suites verifying tier transitions and currency rounding edge cases. |
| **Payment Gateway Timeout / Webhook Failure** | Implement idempotent webhook handlers, automated retry mechanisms, and background reconciliation jobs. |
| **Unauthorized Access to Utility Grid / Customer Data** | Enforce Role-Based Access Control (RBAC), API rate limiting, TLS 1.3 encryption, and OWASP security practices. |

## 10. Conclusion

Applying the structured software project scheduling methodology to the **Utility Management System** ensures a disciplined, transparent, and achievable 15-week delivery roadmap. By leveraging Work Breakdown Structure (WBS), 3-point PERT estimation, and Critical Path Analysis, the project team can systematically mitigate risks, optimize resource utilization, and guarantee high system reliability from requirements gathering through final deployment.

*— End of Report —*